

AI Mode All Images Videos News More 



Ask anything



```
t_size) * 0.01 #  
hidden layer  
en_size) * 0.01 #
```

7/10/25, 18:22

AI Mode

All

Images

Videos

News

More ▾

⚙️

🕒

📝

def tanh(self, x):

return np.tanh(x)

def tanh_derivative(self, x):

return 1 - np.tanh(x)**2

def forward(self, x_t, h_prev):

Hidden state update

h_t = self.tanh(np.dot(self.Wx, x_t) + np.dot(self.Wh, h_prev) + self.bh)

Output calculation

y_t = np.dot(self.Vy, h_t) + self.by

return h_t, y_t

Example usage:

input_size = 10

hidden_size = 20

output_size = 5

learning_rate = 0.01

rnn = RNN(input_size, hidden_size, output_size, learning_rate)

Sample input and previous hidden state

x_t = np.random.rand(input_size, 1) # Current input

h_prev = np.zeros((hidden_size, 1)) # Previous hidden state (initialized to zero)

h_t, y_t = rnn.forward(x_t, h_prev)

print("Current hidden state (h_t):")

print(h_t)

print("\nPredicted output (y_t):")

print(y_t)

Use code with caution.

Explanation

- Initialization:** The constructor initializes the weight matrices (W_x , W_h , V_y) and bias vectors (b_h , b_y) with small random values to prevent issues during training like exploding gradients.
- tanh and tanh_derivative:** These functions implement the hyperbolic tangent activation function and its derivative, which are often used in the hidden layer of RNNs.
- forward pass:**
 - Takes the current input (x_t) and the previous hidden state (h_{prev}).
 - Calculates the new hidden state (h_t) using the weighted sum of the current input, previous hidden state, and bias, then applies the tanh activation function.
 - Calculates the output (y_t) by applying weights and bias to the current hidden state.

This example provides a foundational understanding of a single RNN cell's forward pass. Training an RNN involves iteratively performing forward passes, calculating the loss, and then using backpropagation through time (BPTT) to update the weights and biases based on the error.

AI responses may include mistakes. [Learn more](#)

from mathematics of rnn bptt to pure python

th NumPy)

o train Recurrent Neural
gorithm to account for the
es information across time

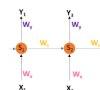
22 sites

Back Propagation through time -
RNN - GeeksforGeeks

May 29, 2025 — Back Propagation
through time - RNN. ... Recurrent...

GeeksforGeeks

Backpropagation Through Time Algorithm -





-
- Ontosight

Backpropagation through time (BPTT) is a gradient-based technique for...

[Show all](#)

Let's consider the previously defined RNN architecture:

- The objective of BPTT is to calculate the gradients of the total loss relative to each of the weight matrices (W_{ij} , U_{ij}) and bias vectors (b_i , c_i). The chain rule will be used to differentiate the loss backward through time.

- ### 1. Gradient of the Loss with respect to the Output Bias:

- ## 2. Gradient of the Loss with respect to the Output Weight:

- ### 3. Gradient of the Loss with respect to the Hidden State:

1. The recurrent nature of the model is apparent here. The hidden state at time t impacts both the output at time t and the hidden state at time $t+1$. Both paths must be considered when calculating the gradient of the loss with respect to $\mathbf{h}_t$.
2. Gradients are backpropagated from the output layer and from the next hidden state.
3. Let δ_t denote $\frac{\partial L}{\partial \mathbf{h}_t}$.
- 4.
- 5.
6. $\delta_t = \delta_t^{\text{out}} + \delta_{t+1}^{\text{hid}} \mathbf{W}_{hh}$ (where $\delta_{t+1}^{\text{hid}}$ is the derivative of the loss with respect to $\mathbf{h}_{t+1}$).

7/10/25, 18:22



1.



6. Gradient of the Loss with respect to the Input Weight:

1.

Pure Python implementation (with NumPy)

python

```
import numpy as np

class RNN:
    def __init__(self, input_size, hidden_size, output_size, learning_rate):
        self.input_size = input_size
        self.hidden_size = hidden_size
        self.output_size = output_size
        self.learning_rate = learning_rate

        # Initialize weights and biases
        self.Wx = np.random.randn(hidden_size, input_size) * 0.01
        self.Wh = np.random.randn(hidden_size, hidden_size) * 0.01
        self.Vy = np.random.randn(output_size, hidden_size) * 0.01

        self.bh = np.zeros((hidden_size, 1))
        self.by = np.zeros((output_size, 1))

        # Memory for gradients (Adagrad-like update)
        self.mWx, self.mWh, self.mVy = np.zeros_like(self.Wx), np.zeros_like(
            self.Wh), np.zeros_like(self.Vy)
        self.mbh, self.mby = np.zeros_like(self.bh), np.zeros_like(self.by)

    def tanh(self, x):
        return np.tanh(x)

    def tanh_derivative(self, x):
        return 1 - np.tanh(x)**2

    def forward(self, inputs, h_prev):
        # Store intermediate values for backpropagation
        xs, hs, os = {}, {}, {}
        hs[-1] = np.copy(h_prev)

        for t in range(len(inputs)):
            xs[t] = inputs[t].reshape(-1, 1) # Ensure inputs are column vectors
            # Hidden state calculation
            hs[t] = self.tanh(np.dot(self.Wx, xs[t]) + np.dot(self.Wh, hs[t-1]) + self.bh)
            # Output calculation
            os[t] = np.dot(self.Vy, hs[t]) + self.by

        return xs, hs, os

    def backward(self, xs, hs, os, targets):
        # Initialize gradients
        dWx, dWh, dVy = np.zeros_like(self.Wx), np.zeros_like(self.Wh), np.zeros_like(
            self.Vy)
        dbh, dby = np.zeros_like(self.bh), np.zeros_like(self.by)

        dh_next = np.zeros_like(hs[0]) # Gradient passed from the next time step

        # Backpropagate through time
        for t in reversed(range(len(xs))):
            # Gradient for output layer
            dy_t = os[t] - targets[t].reshape(-1, 1) # Assuming MSE loss, derivative is (y - target)
            dVy += np.dot(dy_t, hs[t].T)
            dby += dy_t

            # Backpropagate through hidden state
            dhs_t = dVy.dot(self.Vy.T) + dh_next
            dWx += np.dot(dhs_t, xs[t].T)
            dWh += np.dot(dhs_t, hs[t-1].T)
            dbh += dhs_t

            # Combine gradients from previous time step
            dhs_t-1 = dhs_t + self.mWh
            dWx, dWh, dVy, dbh, dby = self.mWx, self.mWh, self.mVy, self.mbh, self.mby + self.learning_rate * (dWx, dWh, dVy, dbh, dby)

        return dWx, dWh, dVy, dbh, dby
```



AI Mode All Images Videos News More



```

        # Clip gradients to mitigate exploding gradients
        for dparam in [dWx, dWh, dVy, dbh, dby]:
            np.clip(dparam, -5, 5, out=dparam) # {Link: according to Analyt.

    return dWx, dWh, dVy, dbh, dby

def update_parameters(self, dWx, dWh, dVy, dbh, dby):
    # Update parameters using Adagrad (or other optimization)
    for param, dparam, mem in zip([self.Wx, self.Wh, self.Vy, self.bh, self.by],
                                   [dWx, dWh, dVy, dbh, dby],
                                   [self.mWx, self.mWh, self.mVy, self.mbh, self.mby]):
        mem += dparam * dparam
        param += -self.learning_rate * dparam / (np.sqrt(mem) + 1e-8)

# Example Usage:
input_size = 3
hidden_size = 5
output_size = 2
learning_rate = 0.01
seq_length = 4

rnn = RNN(input_size, hidden_size, output_size, learning_rate)

# Create a sample sequence of inputs and corresponding targets
inputs = [np.random.rand(input_size, 1) for _ in range(seq_length)]
targets = [np.random.rand(output_size, 1) for _ in range(seq_length)]

# Initialize previous hidden state (e.g., zeros)
h_prev = np.zeros((hidden_size, 1))

# Forward pass
xs, hs, os = rnn.forward(inputs, h_prev)

# Calculate loss (example: sum of MSE)
loss = sum(0.5 * np.sum((os[t] - targets[t].reshape(-1, 1))**2) for t in range(seq_length))
print(f"Initial Loss: {loss:.4f}")

# Backward pass (BPTT)
dWx, dWh, dVy, dbh, dby = rnn.backward(xs, hs, os, targets)

# Update parameters
rnn.update_parameters(dWx, dWh, dVy, dbh, dby)

print("Parameters updated after one BPTT step.")

# You would typically repeat this process for many iterations and sequences
# to train the RNN.

```

Use code with caution.

Key aspects of the Python implementation

1. Forward Pass (`forward` method):

1. This is responsible for calculating the hidden states and outputs for each time step in the sequence.
2. It stores intermediate values (`xs` , `hs` , `os`) in dictionaries for use during backpropagation.

2. Backward Pass (`backward` method):

1. **Initialization:** Gradients for all parameters are initialized to zeros. `dh_next` represents the gradient flowing from the subsequent time step and is initialized to zero for the last time step.
2. **Iterating Backwards:** The core of BPTT is iterating backward through the time steps, starting from the last time step and going back to the first.
3. **Output Layer Gradient:** The gradient of the loss with respect to the output at the time step `t`, this is simply the

hidden state (`dh_t`) is
layer (`dVy.T @ dy_t`) and

propagated through the

- ## Limitations and considerations

- AI responses may include mistakes. [Learn more](#)

×

4	0	7
6	0	9
9	1	1

 Towards AI

Here's the breakdown, building upon the previous RNN class:

Mathematical formulation

Recall the hidden state update equation:

-

To find $\partial h_t / \partial h_{t-1}$, differentiate this equation with respect to h_{t-1} :

- Let
- Then
- Using the chain rule,
- The derivative of $\frac{d}{dt} h_t$ with respect to z is $\frac{d}{dz} h_t$, which is also $\frac{d}{dz} h_{t-1}$.
- So,
- The derivative of $\frac{d}{dt} h_t$ with respect to h_{t-1} is simply $\frac{d}{dt} h_{t-1}$ (since $\frac{d}{dz} h_t = \frac{d}{dz} h_{t-1}$ and $\frac{d}{dt} h_{t-1}$ is constant with respect to h_{t-1}).

BPTT process, needs to
re previous hidden



AI Mode All Images Videos News More



```

import numpy as np

class RNN:
    def __init__(self, input_size, hidden_size, output_size, learning_rate):
        self.input_size = input_size
        self.hidden_size = hidden_size
        self.output_size = output_size
        self.learning_rate = learning_rate

        # Initialize weights and biases
        self.Wx = np.random.randn(hidden_size, input_size) * 0.01
        self.Wh = np.random.randn(hidden_size, hidden_size) * 0.01
        self.Vy = np.random.randn(output_size, hidden_size) * 0.01

        self.bh = np.zeros((hidden_size, 1))
        self.by = np.zeros((output_size, 1))

        # Memory for gradients (Adagrad-like update)
        self.mWx, self.mWh, self.mVy = np.zeros_like(self.Wx), np.zeros_like(self.Wh), np.zeros_like(self.Vy)
        self.mbh, self.mby = np.zeros_like(self.bh), np.zeros_like(self.by)

    def tanh(self, x):
        return np.tanh(x)

    def tanh_derivative(self, x):
        return 1 - np.tanh(x)**2

    def forward(self, inputs, h_prev):
        # Store intermediate values for backpropagation
        xs, hs, os, nets = {}, {}, {}, {} # Add 'nets' to store the pre-tanh activation
        hs[-1] = np.copy(h_prev)

        for t in range(len(inputs)):
            xs[t] = inputs[t].reshape(-1, 1)
            # Calculate the pre-tanh activation
            nets[t] = np.dot(self.Wx, xs[t]) + np.dot(self.Wh, hs[t-1]) + self.bh
            # Hidden state calculation
            hs[t] = self.tanh(nets[t])
            # Output calculation
            os[t] = np.dot(self.Vy, hs[t]) + self.by

        return xs, hs, os, nets

    def backward(self, xs, hs, os, nets, targets): # Include 'nets' in backward pass
        # Initialize gradients
        dWx, dWh, dVy = np.zeros_like(self.Wx), np.zeros_like(self.Wh), np.zeros_like(self.Vy)
        dbh, dby = np.zeros_like(self.bh), np.zeros_like(self.by)

        dh_next = np.zeros_like(hs[0]) # Gradient passed from the next time step

        # Backpropagate through time
        for t in reversed(range(len(xs))):
            # Gradient for output layer
            dy_t = os[t] - targets[t].reshape(-1, 1)

            dVy += np.dot(dy_t, hs[t].T)
            dby += dy_t

            # Gradient for hidden layer
            # Combine gradients from output and next hidden state
            dh_t = np.dot(self.Vy.T, dy_t) + dh_next

            # Backpropagate through tanh
            # This is where we need the derivative of the activation function
            # applied to the pre-activation value (nets[t])
            dh_raw = dh_t * self.tanh_derivative(nets[t])

            dWx += np.dot(dh_raw, xs[t].T)
            dWh += np.dot(dh_raw, hs[t-1].T)
            dbh += dh_raw

```

step

s the gradient *before* the current step
 adient with respect to h_t

AI Mode

All

Images

Videos

News

More

⚙️

🕒

📝

```

        return dWx, dWh, dVy, dbh, dby

    def update_parameters(self, dWx, dWh, dVy, dbh, dby):
        for param, dparam, mem in zip([self.Wx, self.Wh, self.Vy, self.bh, self.bv],
                                       [dWx, dWh, dVy, dbh, dby],
                                       [self.mWx, self.mWh, self.mVy, self.mbh, self.mbv]):
            mem += dparam * dparam
            param += -self.learning_rate * dparam / (np.sqrt(mem) + 1e-8)

# Example Usage:
input_size = 3
hidden_size = 5
output_size = 2
learning_rate = 0.01
seq_length = 4

rnn = RNN(input_size, hidden_size, output_size, learning_rate)

inputs = [np.random.rand(input_size, 1) for _ in range(seq_length)]
targets = [np.random.rand(output_size, 1) for _ in range(seq_length)]

h_prev = np.zeros((hidden_size, 1))

xs, hs, os, nets = rnn.forward(inputs, h_prev) # Get nets from forward pass

loss = sum(0.5 * np.sum((os[t] - targets[t]).reshape(-1, 1))**2) for t in range(seq_length))
print(f"Initial Loss: {loss:.4f}")

dWx, dWh, dVy, dbh, dby = rnn.backward(xs, hs, os, nets, targets) # Pass nets to backward pass

rnn.update_parameters(dWx, dWh, dVy, dbh, dby)

print("Parameters updated after one BPTT step.")

```

Use code with caution.

Key changes and explanation

1. Storing `nets` in the forward pass:

1. The `forward` method now returns `nets` along with `xs`, `hs`, and `os`. The `nets` dictionary stores the *pre-activation* values (`os[t]`) at each time step.
2. This is crucial because the derivative of the activation function (`self.tanh_derivative`) needs to be applied to these pre-activation values during the backward pass.

2. Calculating `dh_next` in the backward pass:

1. `dh_t` represents `dh_t_raw`.
2. `dh_raw` represents the gradient after backpropagating through the `tanh` activation function, i.e., `self.tanh_derivative(nets[t])` (where `dh_t` is `dh_t_raw`).
3. To get the gradient with respect to the *previous* hidden state (`h_prev`), you need to multiply `dh_raw` by the partial derivative of `self.Wh` with respect to `h_prev`, which is `self.Wh.T`.
4. Thus, `dh_next = np.dot(self.Wh.T, dh_raw)` correctly calculates the gradient that will be passed to the next iteration of the backward loop (which corresponds to the previous time step), effectively implementing $\frac{\partial L}{\partial h_{t-1}}$.

This `dh_next` calculation embodies the partial derivative of the current hidden state (`h_t`) with respect to the previous hidden state (`h_{t-1}`) as it propagates the error signal back through the recurrent connection, [according to Towards Data Science](#). It's a fundamental part of how RNNs learn long-term dependencies in sequence data, [says GeeksforGeeks](#).



AI Mode

All

Images

Videos

News

More ▾

